

Quick Summary — Week 3 Important Topics

1) Public key vs private key

A **public key** is openly shared and is used by others to verify signatures or lock value to you. A **private key** is secret and is used by you to sign messages or unlock value that belongs to you. In simple words: public key can be seen by everyone, private key must never be shared.

2) Why Bob cannot just assume received data came from Alice

Just because Bob receives a message does **not** mean Alice truly sent it. Someone else could pretend to be Alice, modify the message, or inject fake data. That is why Bob needs a way to verify both the **sender's identity** and the **message integrity**, which is what digital signatures help do.

3) Naive digital signatures

The naive idea is: Alice signs the whole message using her private key, sends the message plus signature, and Bob verifies using Alice's public key. This seems to prove that Alice sent it and that it was not changed. But this simple-minded version is not the best secure design, which is why the lecture improves it by using **hash before signing**.

4) Hash before signing

Instead of signing the whole message directly, Alice first computes the **hash (digest)** of the message, then signs that digest using her private key. Bob then verifies the signed digest using Alice's public key, computes the hash of the received message himself, and compares both hash values. If they match, the message is authentic and unchanged.

5) Authentication vs integrity

Authentication means proving who sent the message.

Integrity means proving the message was not altered.

A good digital signature system gives both: it shows the message came from the claimed sender, and it shows the contents stayed unchanged.

6) Value locked to public keys in Bitcoin

In Bitcoin, value is thought of as being **locked to a public key**. If value is locked to Alice's public key, then only Alice's matching private key can authorize spending it. So ownership is not just "Alice's account balance" in the normal bank sense — it is cryptographically controlled.

7) Unlocking with private key

To spend value, the owner must prove control using their **private key**. That private key authorizes unlocking previously locked value and lets the user create a new transaction. Then the value is relocked to the next owner's public key.

8) Hard fork vs soft fork

A **hard fork** loosens rules, so some blocks valid under new rules may be invalid under old rules. Old nodes may reject the new chain, so split risk is high unless everyone upgrades. A **soft fork** tightens rules, so new-rule blocks are usually still acceptable to old nodes, making split less likely.

9) Natural/accidental fork vs purposeful fork

A **natural or accidental fork** happens when two miners produce valid blocks at nearly the same time, causing temporary competing branches. These are normal and usually resolve automatically through the longest-chain rule. A **purposeful fork** is an intentional protocol change, like a hard fork or soft fork.

10) 51% attack step by step

In a 51% attack, a malicious miner secretly builds a private chain while honest miners keep building the public one. The attacker spends coins on the public chain but leaves that spending out of the private chain. If the attacker's private chain becomes longer, they publish it, the network accepts it, and the earlier public spending disappears from the accepted history.

11) Double spending through private longer chain

The attacker first spends coins publicly, so the receiver thinks payment happened. But privately, the attacker builds an alternative chain where that payment never happened. Once the private chain becomes longer and is accepted, the attacker still has the coins there and can spend them again — that is double spending.

12) Barter, commodity money, representative money, fiat money

Barter = direct exchange of goods/services without money.

Commodity money = money with intrinsic value, like gold or silver.

Representative money = money that stands for some valuable backing asset, like gold-backed notes.

Fiat money = government-issued money with little or no intrinsic value, accepted because the government declares it legal tender.

13) Intrinsic value vs no intrinsic value

Intrinsic value means the object has value by itself, like gold having material scarcity and usefulness. **No intrinsic value** means the item itself is not valuable mainly because of its material, but because society or government accepts it as money. Commodity money has intrinsic value; fiat money generally does not.

14) 1971 and the end of the US Gold Standard link

The lecture says that in **1971**, the US severed ties with the Gold Standard. This means US money was no longer tied directly to gold backing in the old representative-money sense. That marks the move toward a more purely **fiat money** system.

Your confusion: How does Bob know which hash function to use?

This is a very good question.

The answer is: **the hash function is already part of the agreed protocol/system.**

Bob does not randomly guess the hash function.

Usually one of these is true:

1) The protocol already defines it

In a blockchain or security system, everyone already knows which hash function is being used.

For example:

- Bitcoin uses specific cryptographic algorithms
- a digital signature standard may specify the hash algorithm
- the software/protocol tells all participants what to use

So if Alice and Bob are operating inside the same system, Bob already knows:
“for this system, we hash using SHA-256”
or whatever the defined function is.

2) It may be included as part of the signature scheme/metadata

Sometimes the algorithm is part of the format or specification, so the verifier knows how the signature was created.

3) Both sender and receiver must use the same agreed hash function

Otherwise comparison would fail.

Because if Alice hashes with one algorithm and Bob hashes with another, they will get different digests, and the verification will not work.

Simple story version of the signing process

Here is the full clean version:

1. Alice writes message **M**
2. Alice computes **H(M)** using the agreed hash function
3. Alice signs that hash using her **private key**
4. Alice sends **message M + signature**
5. Bob receives both
6. Bob uses Alice's **public key** to verify/recover the signed digest
7. Bob also computes **H(M)** himself using the **same agreed hash function**
8. Bob compares the two digest values
9. If they match, Bob concludes:
 - message was not changed
 - signature matches Alice's key
 - message is authentic and intact

Super simple answer to memorize for your confusion

Bob knows the hash function because it is predefined by the protocol or signature scheme. Both sender and receiver must use the same agreed hash function for verification to work.

One more subtle thing

In real cryptography, we often say “**verify the signature**” rather than casually saying “decrypt with public key,” because digital signatures are not exactly the same as normal encryption/decryption in every implementation.

But for your course slides right now, the classroom explanation is basically:

- Alice signs with private key
- Bob checks with public key
- Bob hashes the received message using the same agreed hash function
- then compares

So for your midterm, stick to that lecture logic unless sir asks for more technical precision.